

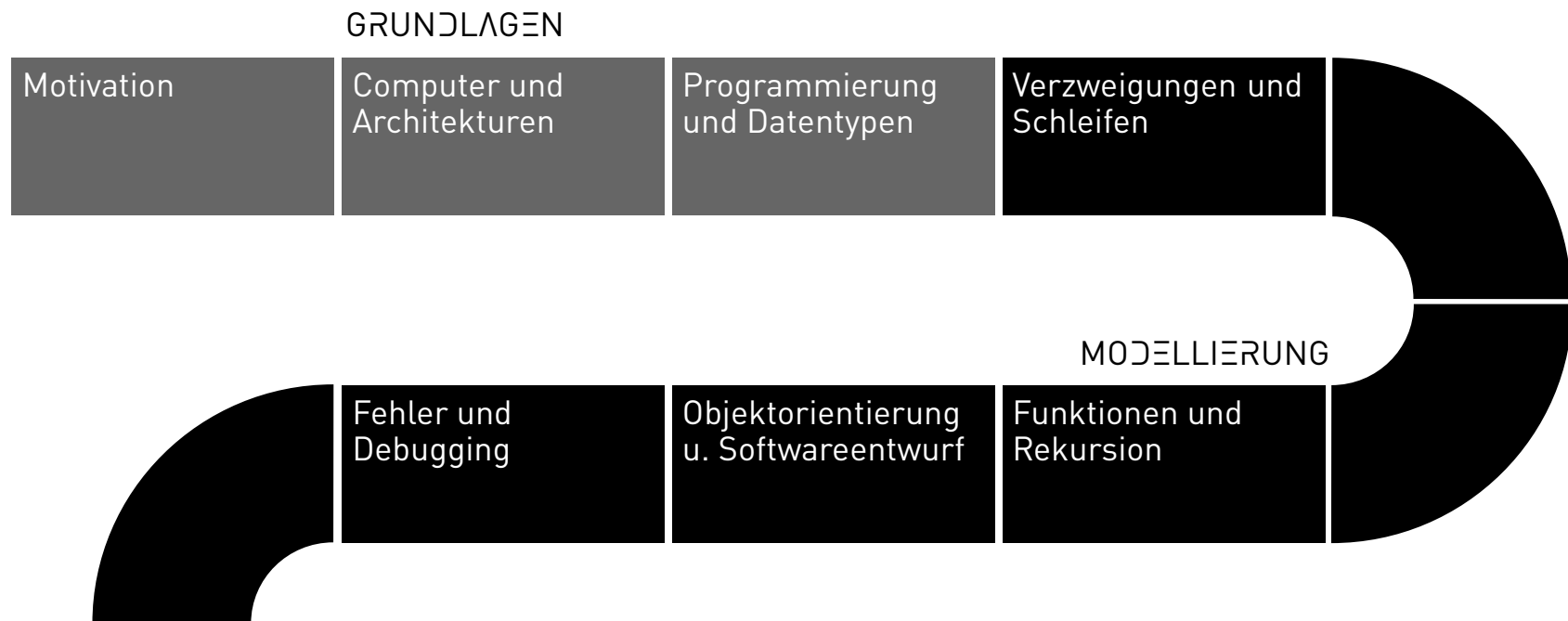
programming and databases

Joern Ploennigs

Data types

MIDJOURNEY: DATATYPE JUNGLE, REF. HENRI ROUSSEAU

PROCESS



VARIABLES

Definition: Variables

In programming, a variable is a value that arises during the execution of a computer program and can usually be changed. A variable is normally identified in source code by a name, has a data type, and an address in the computer's memory.

Definition: Constant

A constant is a value that cannot be changed once it has been assigned.

VARIABLES IN PYTHON

Variables in Python do not need to be explicitly declared (as in many other programming languages); instead, they are assigned a value using the assignment operator (=).

Example:

```
a = 1 # a has the value 1  
  
a = 2 # a now has the value 2  
  
a = "test" # a now has the value "test"
```

The type of a variable can be queried with the function `type(a)`.

VARIABLE NAMES

- Use names that clearly convey the meaning and content of the variables.
- Avoid overly generic names like 'data' or 'v'.
- Keep the naming conventions in your code consistent.
- Use lowercase letters with underscores (_) to separate words (snake_case), e.g., `my_variable`.
- Avoid special characters such as ä, ö, ü, ß, as these can cause problems with character encoding.
- Do not use reserved keywords of the programming language as variable names (e.g., `if`, `for`, `while`).
- Avoid abbreviations or acronyms, unless they are generally understood (e.g., 'GDP').
- Strive for a balance between clarity and brevity.

EXAMPLES OF GOOD AND BAD VARIABLE NAMES

Good	Bad	Description
age	a	Clearly represents a person's age.
first_name	fn	Clearly describes the first name.
birth_year	by	Clearly and unambiguously indicates the birth year.
email_address	email	Makes it clear that this is an email address.
is_on	on	is_ indicates that this is a boolean value.
product_list	products	Makes it clear that this is a list.
score_total	score	Shows that this is the total sum.
user_count	count	Indicates that this is the number of users.

DATA TYPE

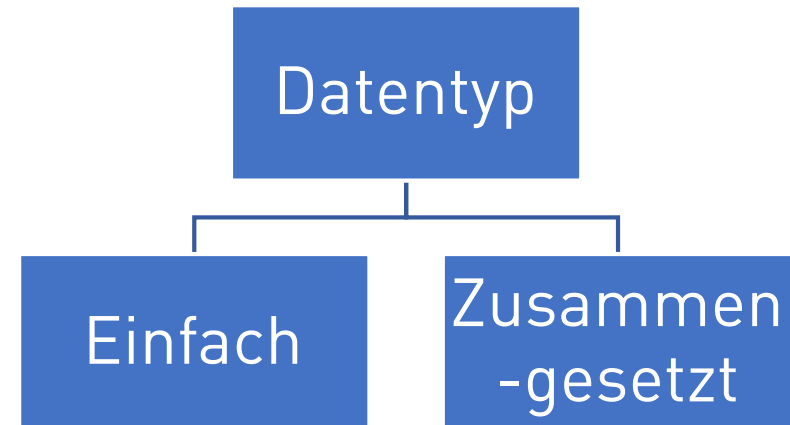


Definition: Data Type

The data type specifies the kind of data described by it and the operations that can be performed on that data.

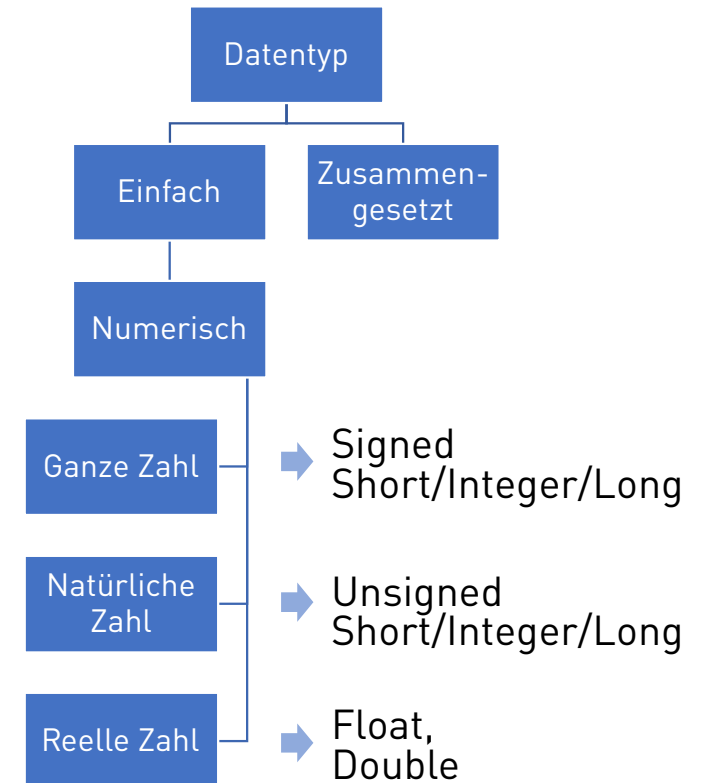
DATA TYPES - STRUCTURE

- Simple data types (primitive data types) can only hold a single value within the corresponding value range.
- Composite data types (complex data types) are data constructs that consist of simpler data types. Since they can theoretically become arbitrarily complex, they are often counted among data structures.



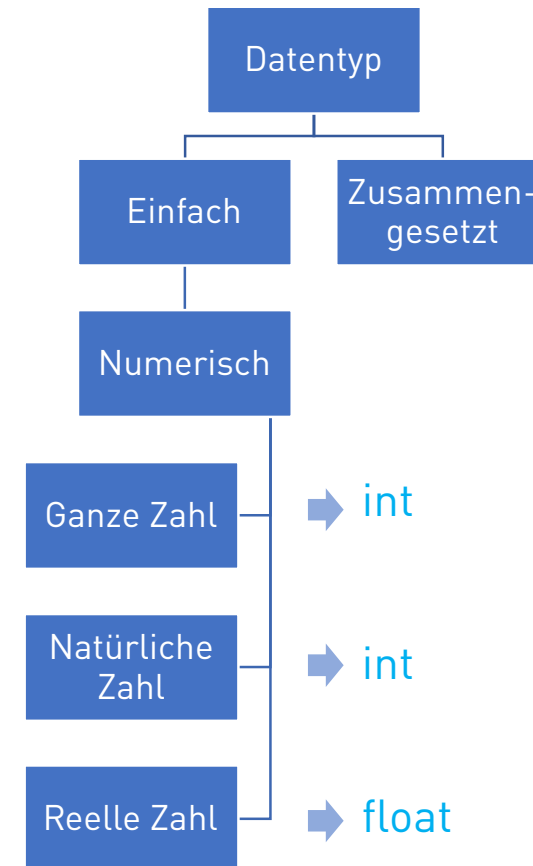
PRIMITIVE DATA TYPES - NUMERIC

- Numeric data types represent numbers.
- Integers and natural numbers are represented as signed and unsigned integers. Depending on storage capacity, the following sizes are distinguished:
 - Short integer (8-bit),
 - Integer (32-bit) and
 - Long (64-bit).
- Real numbers are represented as floating-point numbers
 - Float (32-bit) or
 - Double (64-bit)
- These variants are named differently in different programming languages.



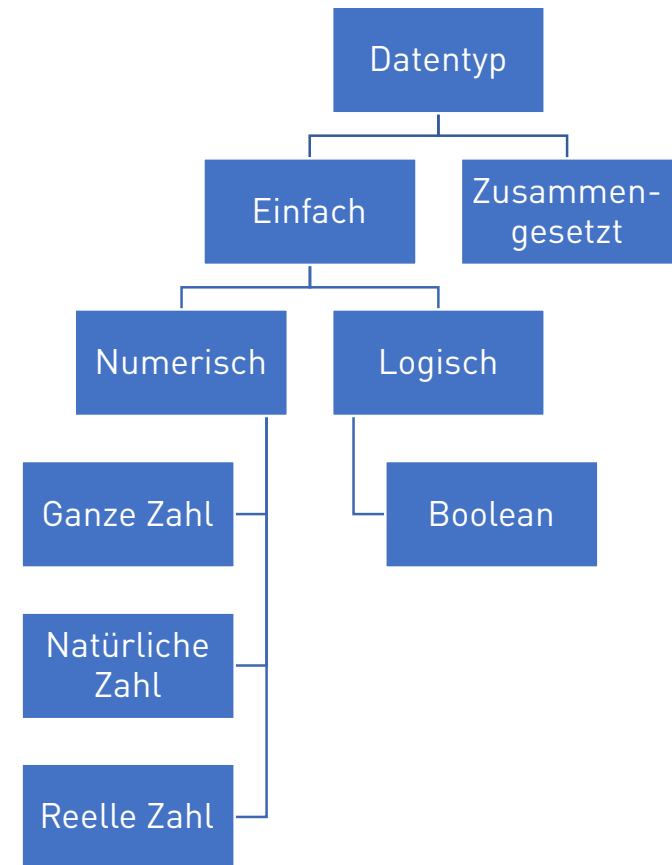
SIMPLE DATA TYPES – NUMERIC IN PYTHON

- Integers and natural numbers are represented in Python by the integer type `int`, and no distinction is made between them.
- Real numbers are represented as the floating-point type `float`.
- **Note:** Cython, a typed variant of Python that compiles to C (i.e., not interpreted), differentiates between `(signed)` `short/int/long` types and `float/double` to run faster.



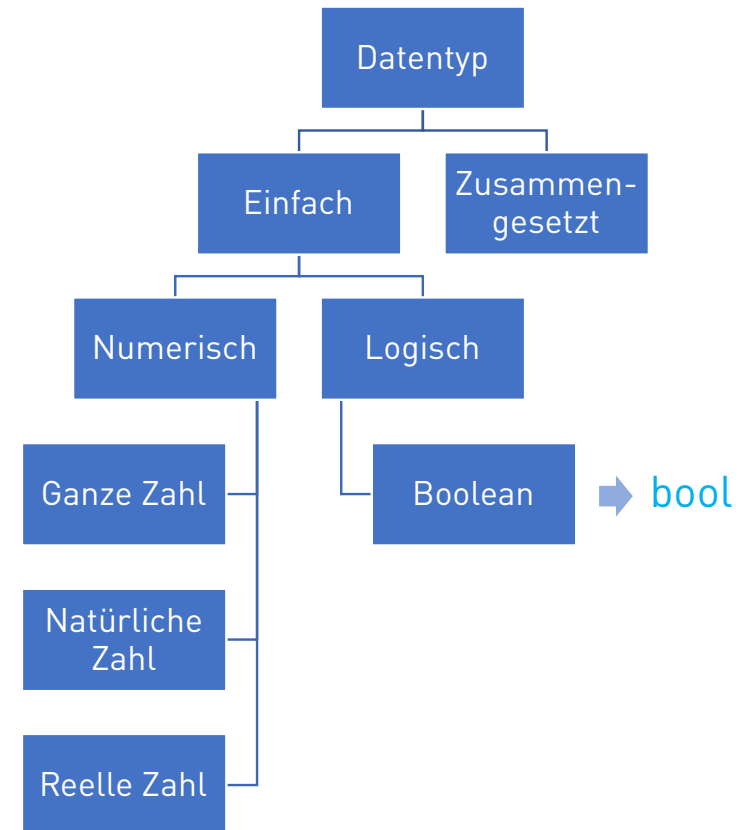
BASIC DATA TYPES - BOOLEAN

- Boolean data types represent binary values such as true or false.
- Truth values are referred to as boolean values.



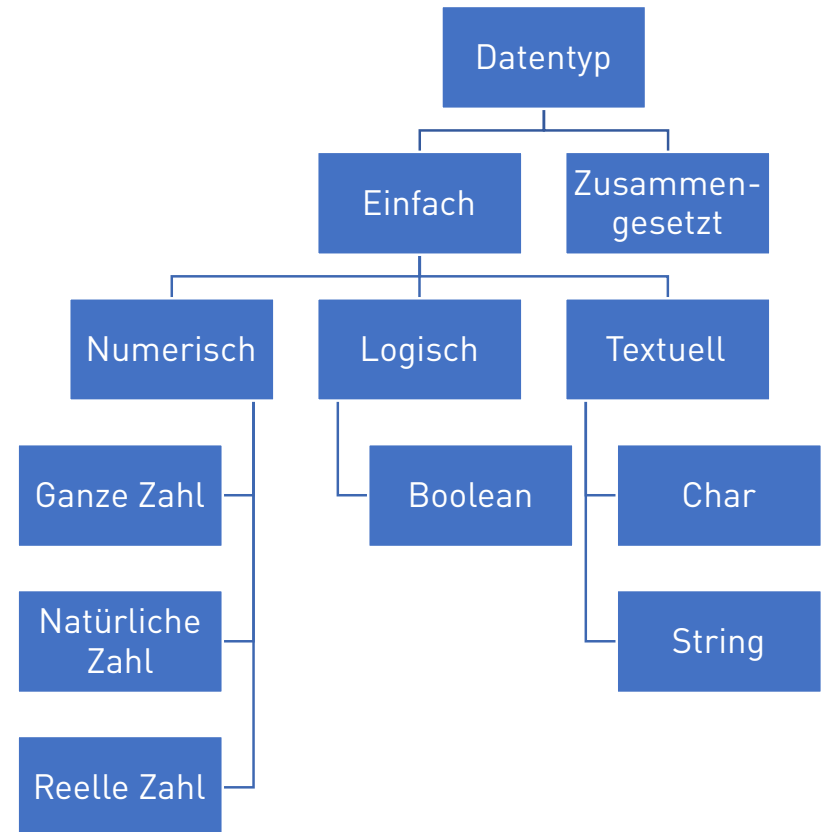
SIMPLE DATA TYPES – BOOLEANS IN PYTHON

- In Python, boolean (binary) values are called `bool`.
- The value 'True' is written in Python as `True` and 'False' as `False`.



SIMPLE DATA TYPES - TEXTUAL

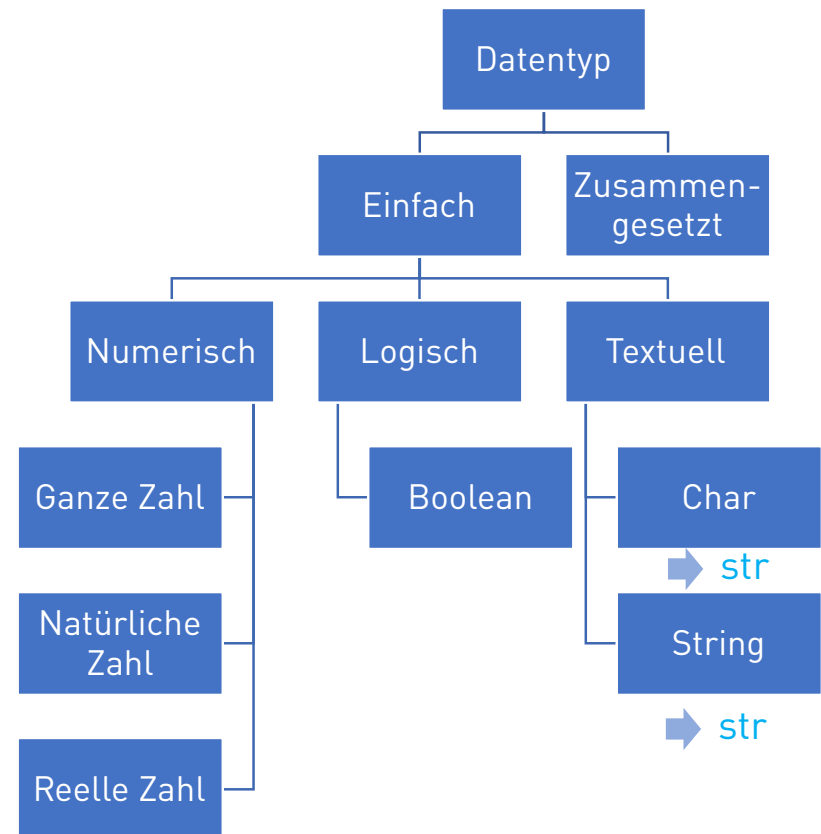
- Textual data types represent letters.
- A single textual character is called a char.
- A string is a sequence of textual characters (it is sometimes also counted among the composite data types).



SIMPLE DATA TYPES – TEXTUAL DATA IN PYTHON

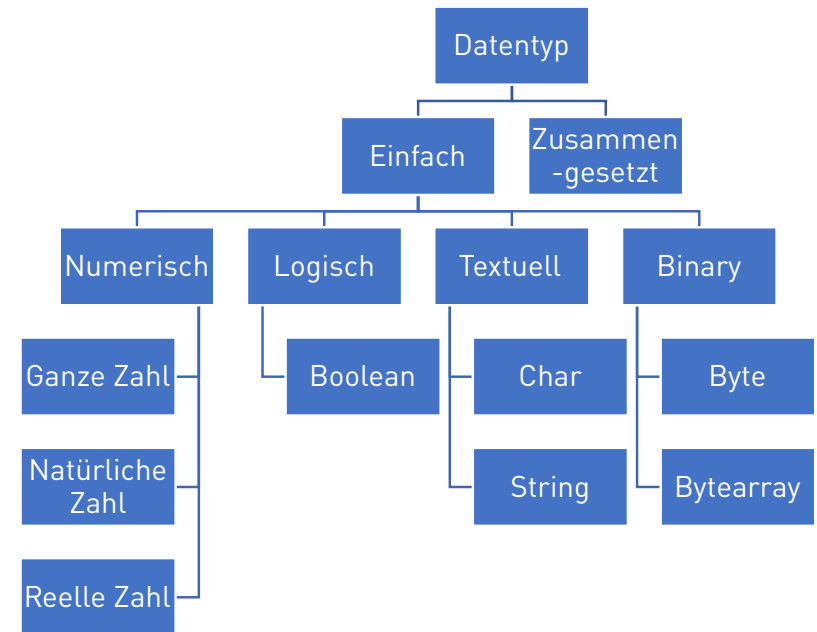
- Individual textual characters are represented in Python as `str` with length 1.
- Multiple textual characters are defined in Python as `str`.
- A `str` in Python can be started and ended with either a `'` or a `"` quotation mark.

```
name = "Joern"  
name = 'Joern'
```



PRIMITIVE DATA TYPES - BINARY

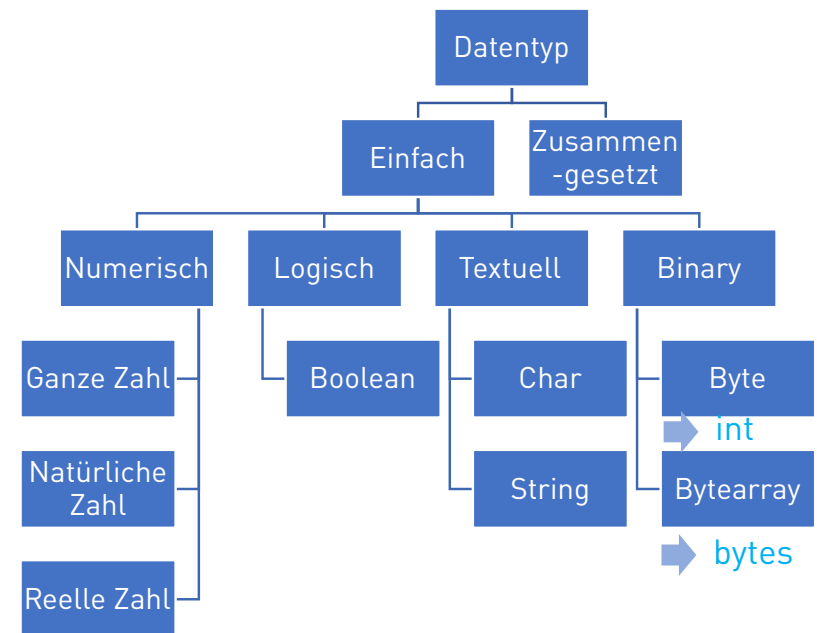
- Binary data types can represent arbitrary characters.
- A single binary character is called a byte.
- Several characters are referred to as a byte array or byte string (they are also sometimes counted among the composite data types).



SIMPLE DATA TYPES – BINARY IN PYTHON

- Binary data types can represent arbitrary characters.
- Individual binary characters are represented in Python as `int`.
- Multiple binary characters are referred to as `bytes`.
- A bytes object in Python is declared as a string with a leading `b`.

```
name = b"Joern"  
name = b'Joern'
```



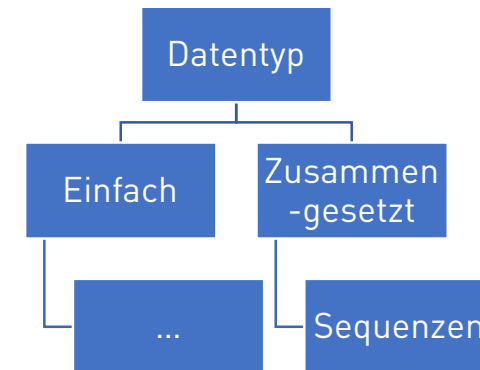
COMPOSITE DATA TYPES – SEQUENCES

- Sequences are an ordered collection of values, usually of the same data type.
- Sequences are typically referred to as arrays in programming languages. Arrays often have a fixed, immutable length defined at creation. The values are mutable.

Example in Cython:

```
cdef int a = 5 # Variable  
declaration  
cdef int a[5] = [0, 1, 2, 3, 4] #  
Array declaration
```

- Lists are another common data type for sequences. Lists often have no fixed length and can be extended arbitrarily.



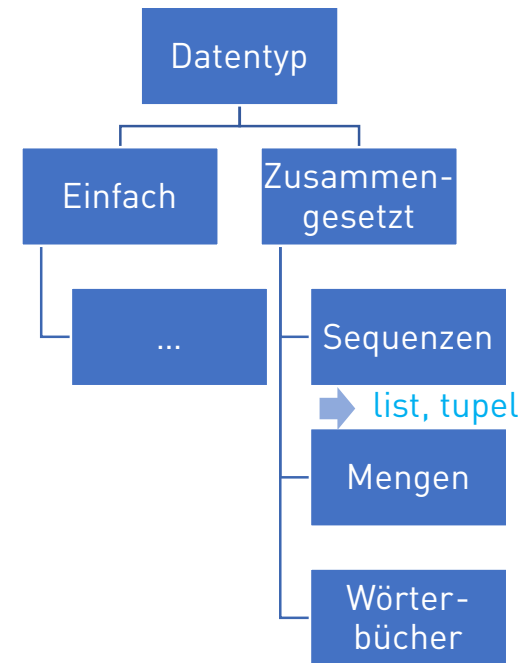
COMPOSITE DATA TYPES – SEQUENCES IN PYTHON

- Python doesn't have arrays; it uses `list` and `tuple`. They are declared using square or round brackets.

```
x = [0, 1, 2, 3, 1] # List  
x = (0, 1, 2, 3, 1) # Tuple
```

- `tuple`s have a fixed length in Python. They are immutable.
- Additionally, there is the special data type `range` to generate a sequence of integers.

```
x = range(0,10) # Range function
```



COMPOSITE DATA TYPES – SEQUENCES: ACCESSING ELEMENTS

- To access an element, the index is written in square brackets.
- In Python, the index in a list starts at 0 (starting at 1 in some languages).

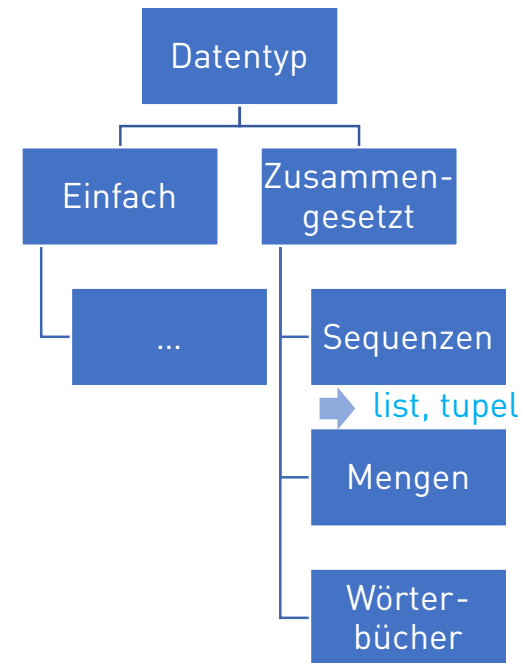
```
x[0] # 1st element  
x[1] # 2nd element
```

- A peculiarity in Python is that negative indices are allowed to access the end of lists (a.k.a. syntax sugar)

```
x[-1] # last element  
x[len(x) - 1] # alternative  
approach
```

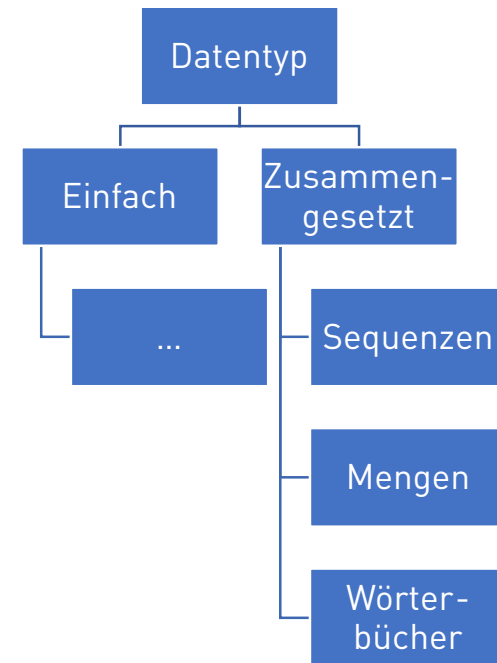
- In Python, slicing can also be used to access portions of lists

```
x[0:10] # first 10 elements (not  
including 10)  
x[:10] # first 10 elements (not  
including 10)  
x[-10:] # last 10 elements
```



COMPOSITE DATA TYPES – SETS

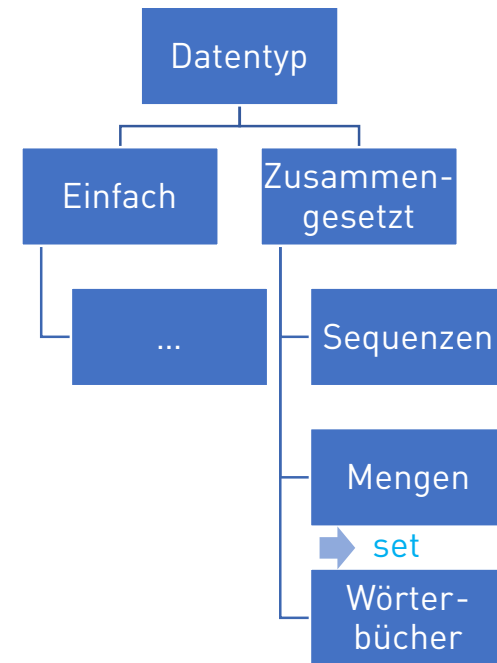
- Sets represent a collection of values without duplicates, just like in mathematics.
- In most programming languages, sets are simply called "sets".
- There is often also a distinction between data types for ordered and unordered sets.



COMPOSITE DATA TYPES – SETS IN PYTHON

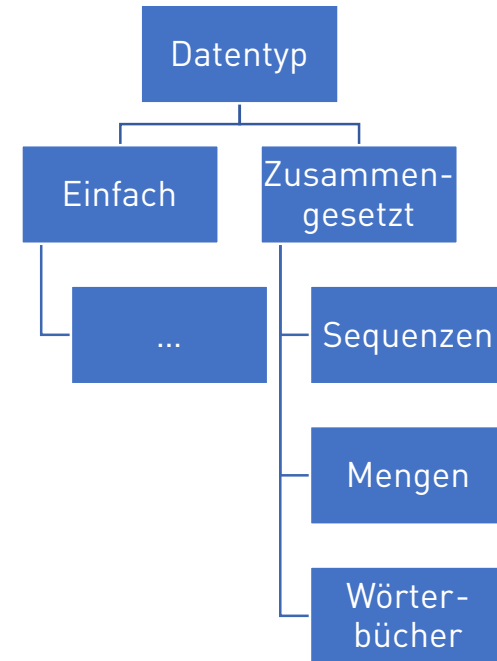
- The data type for sets in Python is called `set`. They are declared using curly braces.

```
x = {0, 1, 2, 3}
```



COMPOSITE DATA TYPES – DICTIONARIES

- Dictionaries map a set of keys to a set of values (Key-Value). The set of keys must not contain duplicates; the set of values may.
- Dictionaries are called maps in most programming languages (from English: Mapping = Abbildung).
- **Info:** Sets are often stored internally as maps without values, because keys must not be duplicated.



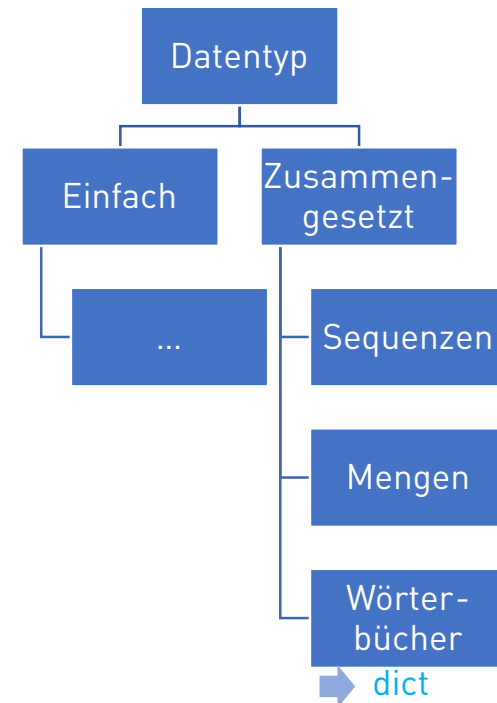
COMPOSITE DATA TYPES – DICTIONARIES IN PYTHON

- Dictionaries map a set of keys to a set of values (key-value pairs).
- Dictionaries in Python are called `dict`. They are defined using curly braces and key/value pairs.

```
x = {  
    "Building type": "Residential  
building",  
    "Year built": 2022  
}
```

- New values can also be assigned dynamically:

```
x["Construction method"] = "Timber  
construction"
```



COMPOSITE DATA TYPES – DICTIONARIES: ACCESSING ELEMENTS

- To access an element in a `dict`, the key is written inside square brackets.

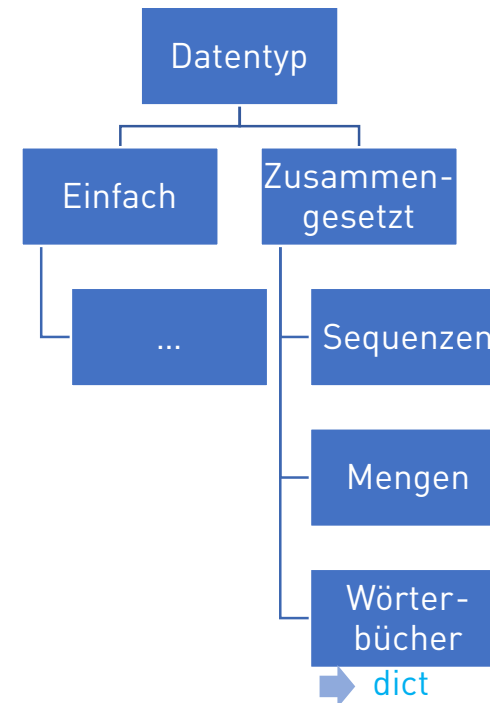
```
x["Baujahr"]
```

- This does not work with sets, since there is no value behind it, only whether the key is contained in the set. This is how you check it

```
"Baujahr" in x
```

- To remove values from a `dict` or `set`, you use

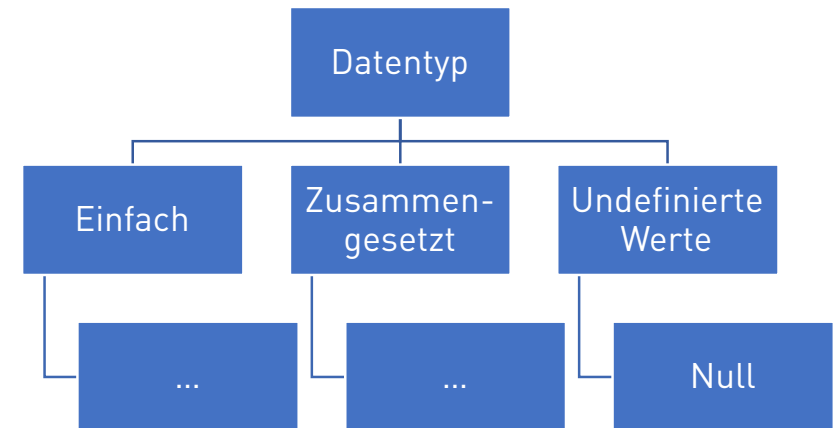
```
del x["Baujahr"]
```



DATA TYPES – UNDEFINED VALUES

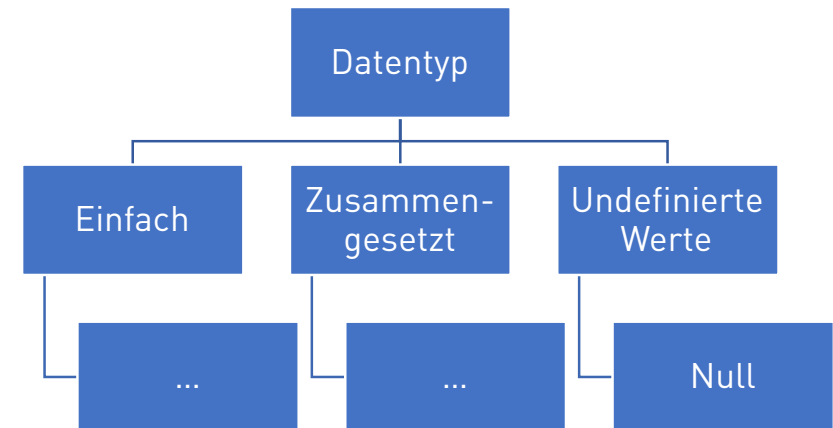
- In many programming languages there is also a value to represent an undefined value, for example when something is not present.
- This undefined value is often referred to as the `null` value.

The need for a null value is nowadays highly controversial, because null values can easily cause errors. Therefore some modern languages do not have a null value.



DATA TYPES – UNDEFINED VALUES IN PYTHON

- The null value in Python is called `None`.
- This means that a variable has not been assigned a value, or that an operation does not return a value.
- The data type of a variable with the value `None` is `NoneType`.



DATA TYPES – MUTABILITY

Definition: Mutability

Mutability describes the ability to modify data structures.

- If a data type is mutable, variables of this type can be modified directly.
- If it is immutable, you can only change them by performing a complete reassignment.

Mutable (changeable)	Immutable (not changeable)
<code>list</code>	<code>tuple</code>
<code>set</code>	<code>frozenset</code>
<code>dict</code>	<code>frozendict</code>
<code>bytearray</code>	<code>bytes</code>

- To prevent programming errors and ensure access security (data protection), some programming languages distinguish very strictly between mutable and immutable data types.

Questions?

